

Senior-Level MVC Interview Questions & Answers

1. Explain the MVC architecture and its advantages.

Answer:

MVC (Model-View-Controller) separates concerns:

- **Model:** Handles business logic and data.
- **View:** Handles UI rendering.
- **Controller:** Handles user input and application flow.

Advantages:

- Separation of concerns (easier testing and maintenance).
 - Supports parallel development (UI and business logic separately).
 - Encourages clean code and reusability.
-

2. What is the difference between TempData, ViewData, and ViewBag?

Answer:

- **ViewData:** Dictionary of key-value pairs, requires type casting, available only during the current request.
 - **ViewBag:** Wrapper around ViewData, uses dynamic type (no casting required).
 - **TempData:** Stores data for the next request (uses session internally). Useful for redirects.
-

3. How does Routing work in MVC?

Answer:

- ASP.NET MVC uses **Routing** to map incoming URLs to controller actions.
 - Configured in RouteConfig.cs (e.g., routes.MapRoute("Default", "{controller}/{action}/{id}")).
 - It avoids hard-coded URLs and provides SEO-friendly URLs.
 - Supports **Attribute Routing** (e.g., [Route("products/{id}")]).
-

4. What is the difference between ActionResult and JsonResult?

Answer:

- **ActionResult:** Base class for all results returned by a controller (e.g., ViewResult, RedirectResult, FileResult).
 - **JsonResult:** A specific type of ActionResult used to return JSON-formatted data (useful in AJAX calls).
-

5. What are Filters in MVC and their types?

Answer:

Filters are attributes that provide cross-cutting concerns. Types:

- **Authorization filters** ([Authorize]) – for authentication/authorization.
 - **Action filters** – run before/after an action (OnActionExecuting).
 - **Result filters** – run before/after a view result (OnResultExecuting).
 - **Exception filters** – handle exceptions globally (HandleErrorAttribute).
-

6. What is the difference between Partial View and Layout View?

Answer:

- **Partial View:** Reusable UI component (e.g., header, menu, comments section). Doesn't include _ViewStart.cshtml.
 - **Layout View:** Defines a common layout (like master page), used across multiple views.
-

7. How can you implement Dependency Injection in ASP.NET MVC?

Answer:

- ASP.NET MVC supports DI via constructors, property injection, or action injection.
 - Third-party containers like **Unity**, **Autofac**, **Ninject** can be integrated.
 - In .NET Core MVC, built-in **IoC container** (ConfigureServices in Startup.cs).
-

8. What are Bundling and Minification in MVC? Why are they useful?

Answer:

- **Bundling:** Combines multiple CSS/JS files into one file → reduces HTTP requests.
 - **Minification:** Removes unnecessary characters (spaces, comments) → reduces file size.
 - Improves **page load performance** and **SEO ranking**.
-

9. How does MVC handle Session and State Management?

Answer:

- **Session State:** Stores user data across requests (in-process, State Server, SQL Server, distributed cache).
 - **TempData:** Short-lived storage for redirection scenarios.
 - **Cookies:** Lightweight, stored on client-side.
 - **Caching:** Page, data, and output caching for performance.
-

10. How would you secure an ASP.NET MVC application?

Answer:

- Use **[Authorize]** attribute for authentication/authorization.
 - **Anti-forgery tokens** (@Html.AntiForgeryToken()) to prevent CSRF attacks.
 - **ValidateInput(false)/RequestValidation** → carefully handle XSS.
 - **HTTPS/SSL** for data security.
 - **OWASP recommended practices** (input validation, output encoding, secure cookies).
-

SET 2

1. Explain the MVC pattern and its benefits.

Answer:

MVC (Model–View–Controller) separates the application into:

- **Model** → Represents the application's data and business logic.
- **View** → Handles the UI representation.
- **Controller** → Processes incoming requests, interacts with the model, and selects the view to render.

Benefits:

- **Separation of Concerns** → Easier maintenance and scalability.
 - **Testability** → Business logic is isolated, making unit testing simpler.
 - **Parallel Development** → UI and business logic can be developed independently.
 - **Extensibility** → Easy to swap out Views or Models without impacting others.
-

2. How does the request life cycle work in ASP.NET MVC?

Answer:

1. **Request enters Routing module** (RouteTable, RouteConfig).
 2. **Routing selects Controller & Action** based on URL pattern.
 3. **Controller Action executes** → interacts with Model.
 4. **Controller passes data to View** (via ViewData, ViewBag, TempData, or strongly typed model).
 5. **View Engine (Razor)** renders the View.
 6. **Response sent back to the browser.**
-

3. What is the difference between TempData, ViewData, and ViewBag?

Answer:

- **ViewData** → Dictionary (string-object) for passing data from Controller to View (works only for the current request).
 - **ViewBag** → Dynamic wrapper around ViewData (syntactic sugar, works for current request).
 - **TempData** → Uses Session under the hood, persists data between two consecutive requests (redirect scenarios).
-

4. What are Action Filters? Can you give a real-world example where you implemented one?

Answer:

Action Filters are attributes that allow you to execute code before or after controller actions. Examples:

- **[Authorize]** → Authorization checks.
- **[HandleError]** → Exception handling.
- **Custom Filters** → For logging, auditing, performance profiling.

Example from a project:

I implemented a custom **AuditLogFilter** to automatically log user actions into the database whenever a critical controller action was executed. This avoided writing repetitive logging code in each action.

5. What's the difference between Partial View and RenderPartial?

Answer:

- **Partial View** (Html.Partial) → Returns an HTML string that you can embed anywhere in the view.
 - **RenderPartial** (Html.RenderPartial) → Writes HTML directly to the Response stream (slightly faster, but doesn't return a value).
-

6. How do you implement Dependency Injection in MVC?

Answer:

- Use an IoC container (Autofac, Unity, Ninject, built-in .NET Core DI in newer projects).
- Register services in Global.asax or Startup.
- Controllers accept dependencies via constructor injection.

Example:

```
public class HomeController : Controller
{
    private readonly ICustomerService _service;

    public HomeController(ICustomerService service)
    {
    }
```

```
_service = service;  
  
}  
  
}
```

7. How do you secure an MVC application?

Answer:

- Use **[Authorize]** for role-based or claims-based access.
 - Apply **Anti-forgery tokens** for form submissions.
 - Always validate input to prevent SQL Injection / XSS.
 - Use HTTPS and secure cookies.
 - Implement output caching carefully to avoid caching sensitive data.
-

8. Explain Razor View Engine's advantages.

Answer:

- Clean, concise syntax (@ for inline C#).
 - Automatic HTML encoding (prevents XSS).
 - Supports IntelliSense in Visual Studio.
 - Combines C# and HTML seamlessly.
-

9. How do you handle large data in MVC without performance issues?

Answer:

- Use **Paging & Lazy Loading**.
 - Implement **Output Caching** for repeated requests.
 - Optimize database queries (use projections, stored procedures).
 - Consider **AJAX partial loading**.
 - Use **Asynchronous Controllers** (async/await) for long-running operations.
-

10. Have you integrated MVC with Web API in a project? How?

Answer:

Yes. In one project, I used MVC for HTML views and Web API for mobile app & AJAX calls:

- Both shared the same solution, with Web API controllers for JSON responses.
- Used RouteConfig for MVC and WebApiConfig for API endpoints.
- Secured both using JWT tokens and role-based authorization.